

Project Proposal By :



MORASHIFT

UNIMED

A CENTRALIZED UNIVERSITY ELECTRONIC
HEALTH RECORD & INTEGRATED
MANAGEMENT SYSTEM

240456R - Neralampitiya N.P.C.

240177L - Galagedara G.K.A.T.

240336B - Karunathilaka B.K.V.P.

240593H - Sangadith K.G.D.N.

240294R - Jayawardana Y.G.L.S.

1. Project Introduction

UniMed: Voice-Enabled Digital Medical Record System is designed to modernize university medical centers by transitioning from manual, paper-based workflows to a streamlined digital ecosystem. By adopting an Object-Oriented (OO) architecture, the system models patients, medical records, and clinical actions as interacting objects, ensuring data integrity, modularity, and system scalability.

1.1. Project Background

Existing manual systems are highly vulnerable to physical file damage, unauthorized access, and significant administrative delays. UniMed mitigates these risks by digitizing student records and housing them within a secure, centralized database, allowing for rapid retrieval and reliable data persistence.

1.2. System Overview

The architecture of UniMed is categorized into four primary components: Data Objects, User Actors, the Processing Engine, and the Interface.

1.2.1. Data Objects (Entities)

- Student Object: The primary data entity containing attributes such as Index Number, Name, and an encapsulated Collection<MedicalRecord>.
- Medical Record Object: A specialized object containing Diagnosis, Prescription, and Timestamp attributes, linked to a specific Student.

1.2.2. User Actors (Interaction)

- Medical Officer: An authorized actor with the permissions to trigger Retrieve() and Update() methods on Student profiles.
- Administrative Clerk: An actor responsible for the Registration and Instantiation of new Student objects.

1.2.3. System Engine (Utility)

- VoiceRecognition Engine: A specialized utility object that serves as the core processing layer. It captures verbal input, parses natural language, and converts it into structured string data for record fields.

1.2.4. System Interface

- **Voice-to-Text Interface:** The bridge between the Medical Officer and the system, facilitating hands-free data entry and real-time attribute mapping.

1.3. Operational Workflow

- **Registration:** The Administrative Clerk instantiates a new Student Object linked to a unique, immutable Index Number.
- **Retrieval:** The Medical Officer requests the Student Object history via the Index Number to review past clinical interactions.
- **Voice Interaction:** The VoiceRecognition Engine captures verbal diagnosis and prescription details, parsing them into the corresponding object attributes.
- **Data Encapsulation:** The system encapsulates the new visit details into a Medical Record Object, updates the Student's collection, and persists the data to the central database.

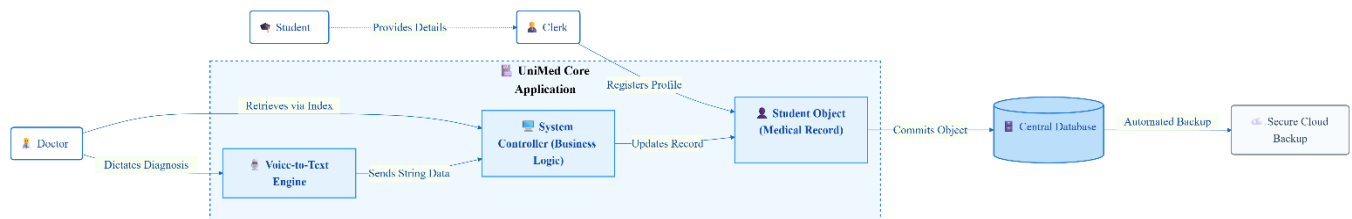


Fig.1

2. Non-Functional Requirements

The following requirements have been refined to support an Object-Oriented architecture, focusing on modularity and system robustness:

2.1. Essential Requirements

- **Security:** The system utilizes object-level access control and data encryption (HTTPS/AES) to protect sensitive Student objects from unauthorized access.

- **Safety & Data Integrity:** High integrity is maintained through encapsulation, ensuring that medical record attributes cannot be corrupted by external processes.
- **Reliability:** The system objects must remain instantiated and available throughout medical center operating hours.
- **Accuracy:** The speech-to-text engine is tuned for high precision in medical terminology to ensure the data stored in the objects is medically valid.

2.2. Quality of Service

- **Responsiveness:** The "Retrieve" method must return a Student object within 2 seconds of the Index Number query.
- **User-friendliness:** The interface objects (buttons, text fields) are designed for high contrast and simplicity, requiring minimal training for staff.

2.3. Future Scalability

- **Portability:** The system is designed with a decoupled architecture, allowing the UI objects to be adapted for mobile devices in the future.
- **Extensibility:** Through inheritance, the system can later support multi-language modules (e.g., Sinhala) without altering the core business logic.

3. Technical Analysis & UML Modeling

The architectural behavior of UniMed is modeled using the Unified Modeling Language (UML) to define precise object interactions and system states.

3.1. Use Case Diagram

The system identifies four primary actors interacting with the UniMed boundary:

- **Medical Staff (Clerk):** Responsible for the "Register New Student Profile" use case.
- **University Medical Officer (Doctor):** Engages in "Retrieve Medical History," "Input Diagnosis via Voice-to-Text," "Manually Edit Converted Text," and "Save Visit Details".

- System Administrator: Manages the "Backup Central Database" use case to ensure data persistence.
- Student: Serves as the primary subject of the records within the system.

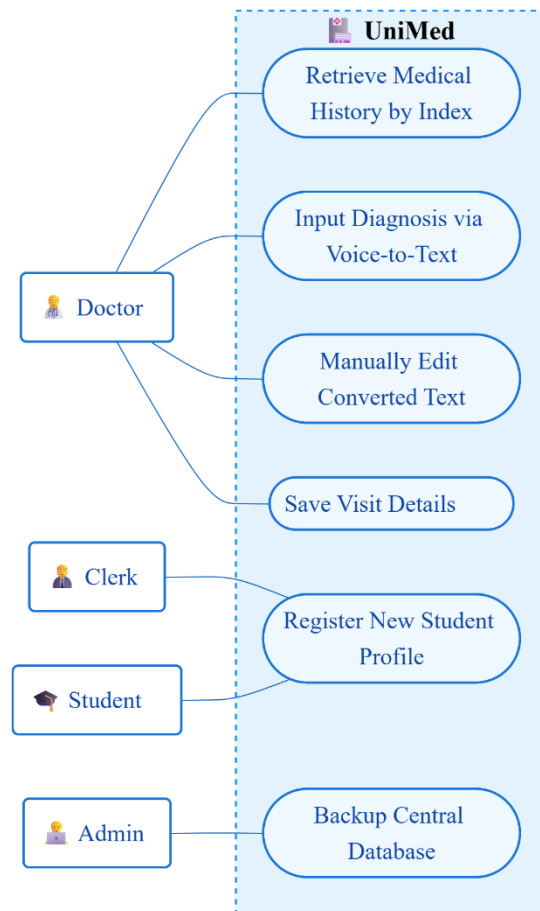


Fig.2

3.2. Activity Diagram (Consultation Flow)

This diagram tracks the logic of a single patient encounter:

- Initialization: The process starts by entering the Student Index Number.
- Decision Logic: If the record is not found, the system prompts the Clerk for registration.
- Voice Integration: Upon finding a record, the system activates the Voice-to-Text interface and converts speech to text.
- Validation: A decision node allows the Doctor to "Manually Edit Text Field" if a correction is needed before saving details to the permanent record.

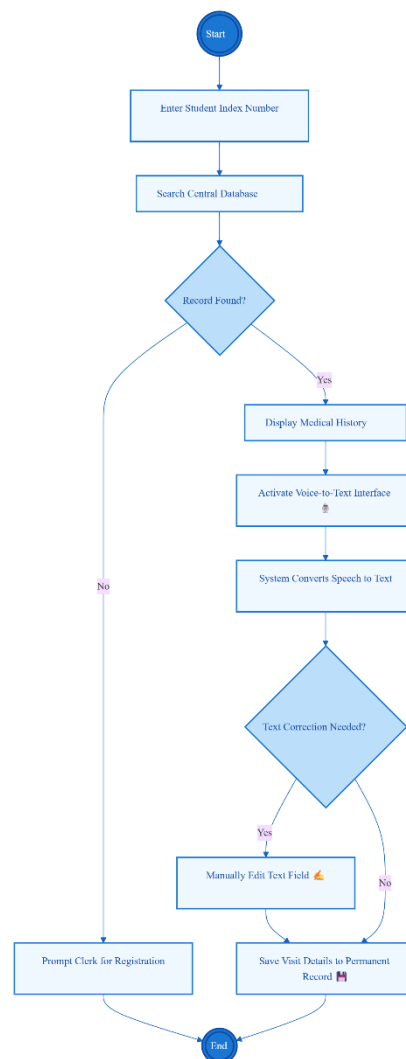


Fig.3

3.3. Sequence Diagram (Object Interaction)

The sequence diagram illustrates the time-ordered message exchange between objects:

1. Request: The Doctor inputs the ID through the ConsultationUI, which triggers `fetchStudent(id)` to the Controller.
2. Data Retrieval: The Controller queries the DatabaseManager, returning a `StudentObject` to the UI.
3. Voice Processing: The Doctor triggers `startVoice()`; the ConsultationUI sends `captureAudio()` to the VoiceEngine, which returns `stringText`.
4. Finalization: After a `confirmAndSave()` call, the Controller sends `save(record)` to the DatabaseManager and returns a success message to the Doctor.

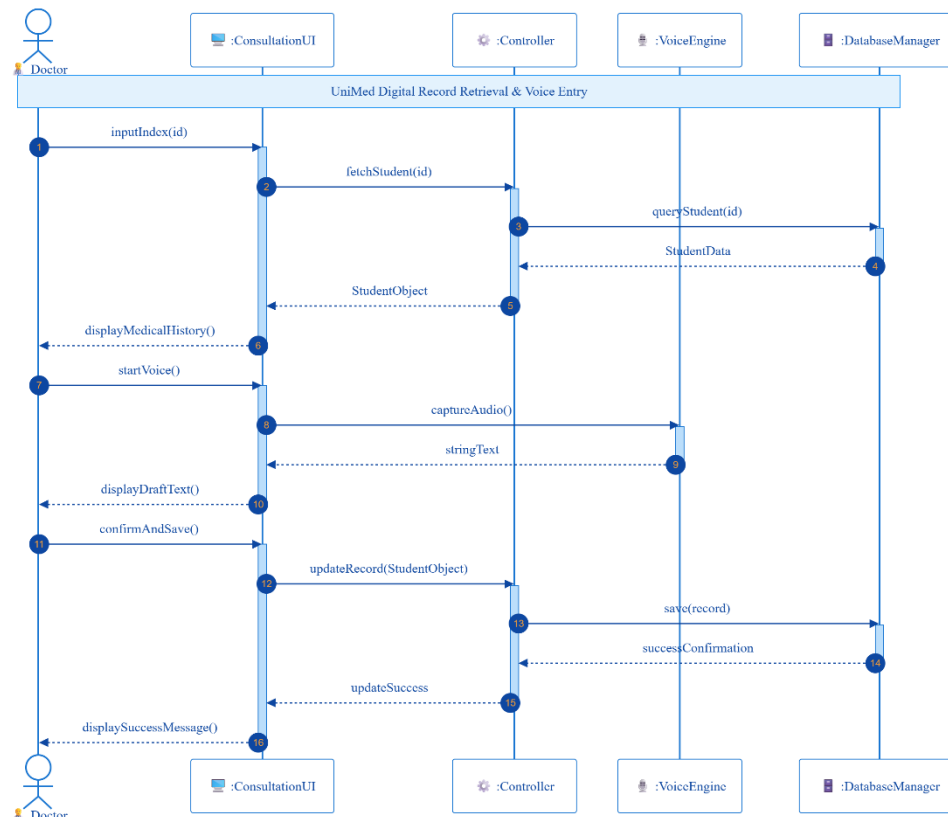


Fig.4

3.4. State Machine Diagram (Record Lifecycle)

The system tracks the lifecycle of a record entry through distinct operational states:

- Initial: The process begins with the "Record entry triggered" state.
- Listening: Triggered by activateVoice(), the system enters an active "Voice-to-Text" state.
- Draft: Once input is received, the record is in "Text Awaiting Review".
- Validated: If the doctor confirms accuracy or performs a manual edit, the state transitions to "Text Confirmed".
- Committed: The final state where the entry is "Saved to Database".

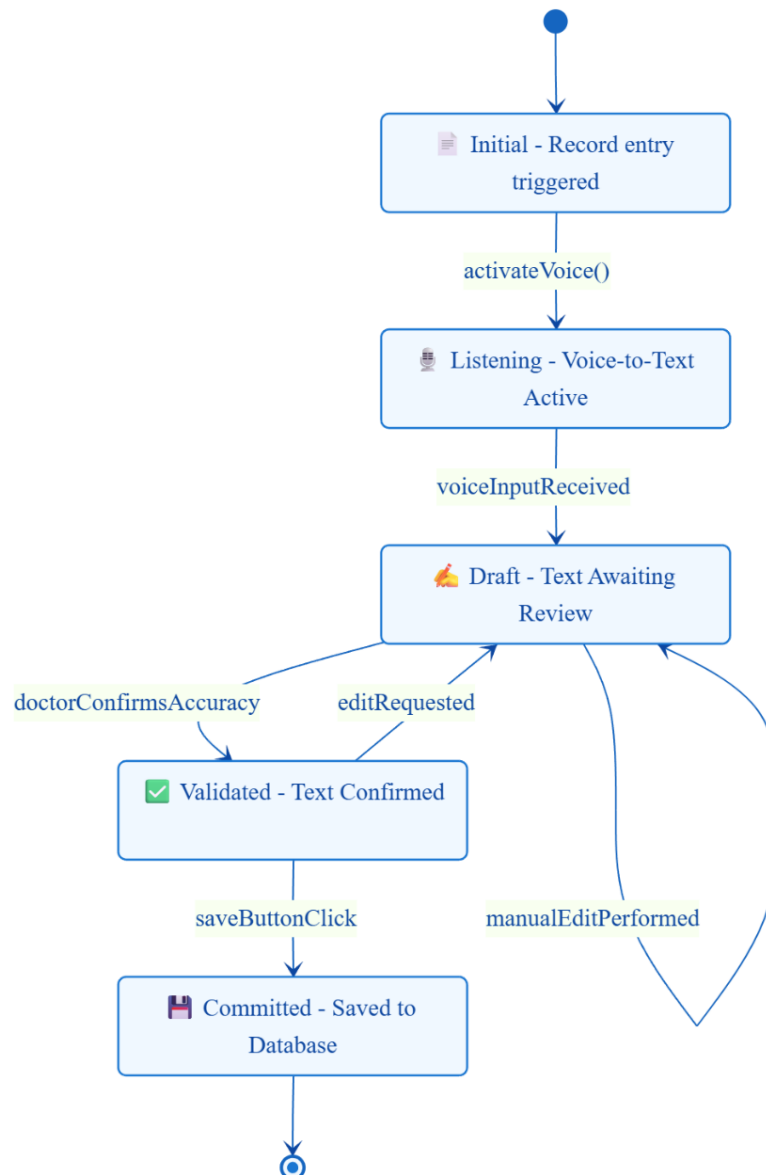


Fig.5

4. References section

- [1] Wazlawick, R. S. (2024). *Object-Oriented Analysis and Design for Information Systems*. Available: https://books.google.lk/books/about/Object_Oriented_Analysis_and_Design_for.html?id=WWv7EAAAQBAJ&redir_esc=y
- [2] Vasilakis, C. (2010). Application of Unified Modelling Language (UML) to the Modelling of Health Care Systems. Available: https://www.researchgate.net/publication/269558239_Application_of_Unified_Modelling_Language_UML_to_the_Modelling_of_Health_Care_Systems
- [3] Jackson, N. (2024). Speech Recognition in Healthcare: Voice-Based Clinical Documentation. Available: https://www.researchgate.net/publication/389497357_Speech_Recognition_in_Healthcare_Voice-Based_Clinical_Documentation
- [4] Alboksmaty, A. (2024). The impact of using AI-powered voice-to-text technology for clinical documentation. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12301838/>
- [5] Westerlund, M., et al. (2025). Artificial Intelligence (AI) – Powered Documentation Systems in Healthcare. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC11835907/>
- [6] GeeksforGeeks. Non-Functional Requirements in Software Engineering. Available: <https://www.geeksforgeeks.org/software-engineering/non-functional-requirements-in-software-engineering/>
- [7] NHS England Digital. GP Connect: Non-functional Requirements for Digital Health Services. Available: <https://digital.nhs.uk/services/gp-connect/develop-gp-connect-services/development/non-functional-requirements>

Images:

Fig.1 : https://mermaid.live/edit#pako:fig.1_Sodmg

Fig.2 : https://mermaid.live/edit#pako:fig.2_Sodmg

Fig.3 : https://mermaid.live/edit#pako:fig.3_Sodmg

Fig.4 : https://mermaid.live/edit#pako:fig.4_Sodmg

Fig.5 : https://mermaid.live/edit#pako:fig.5_Sodmg

Project Proposal By :



MORASHIFT



**THANK
YOU**